

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 40%

Dave's Taxonomy: Precision (Level 3)
Question Count: 5

Total Marks: 40

Code of Conduct

I **K.MAHESH BABU(192525122)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **K.MAHESH BABU(192525122)**

Assessment Tasks

Experiments

1. Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
2. Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
3. Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
4. Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
5. Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

For each experiment, include objective, architecture sketch, procedure, expected output, and conclusion.

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity
Use of Tools /	20%	Appropriate	Good tool	Limited	Incorrect or

Platforms		and effective use of cloud tools	usage with minor issues	tool usage	missing tool usage
Application of Concepts	20%	Concepts are applied accurately to practical tasks	Mostly accurate application	Partial application	Weak application
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

EXPERIMENT 1: CLOUD STORAGE COMPARISON

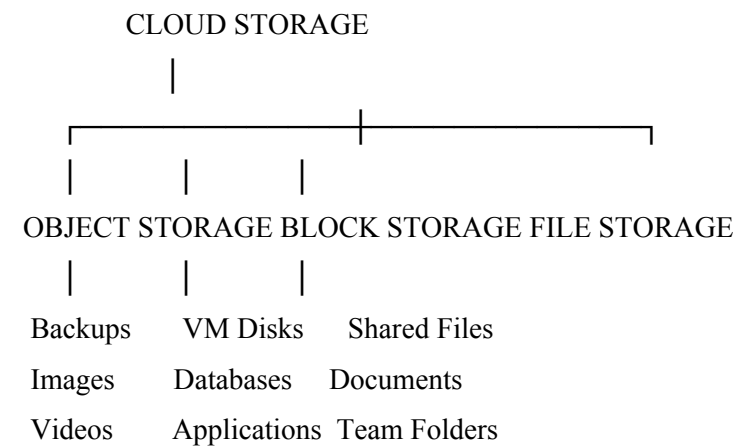
Aim

To create and document a comparison between **object storage, block storage, and file storage**, and select the best storage model for backups, VM disks, and shared enterprise documents.

Objective

- Understand different cloud storage models.
- Compare object, block, and file storage.
- Select a suitable storage model for different workloads.
-

Architecture Sketch



Comparison

Feature	Object Storage	Block Storage	File Storage
Data type	Objects/files	Blocks	Files/folders
Best use	Backups, images, videos	VM disks	Shared documents
Access	API/object interface	Attached volume	File system
Scalability	Very high	High	High
Example	Cloud backup	VM boot disk	Department shared folder

Selection

Requirement	Best Model	Reason
Backups	Object Storage	Suitable for large unstructured data
VM disks	Block Storage	Works as a virtual disk

Shared enterprise documents	File Storage	Supports shared files and folders
-----------------------------	--------------	-----------------------------------

Procedure

- 1. Identify the storage requirements.
- 2. Create a comparison of object, block, and file storage.
- 3. Analyze access method and workload.
- 4. Select the most suitable storage model.
- 5. Document the result.

Result

Object storage is selected for **backups**, block storage for **VM disks**, and file storage for **shared enterprise documents**.

EXPERIMENT 2: REST API WORKFLOW

Aim

To develop a simple **REST API workflow** for a cloud application including authentication and deployment flow.

Objective

- Understand REST APIs.
- Demonstrate client-server communication.
- Understand authentication.
- Understand cloud deployment.

Architecture Sketch

CLIENT / BROWSER

|
| HTTP Request



Authentication



CLOUD APPLICATION



CLOUD DATABASE

REST API Methods

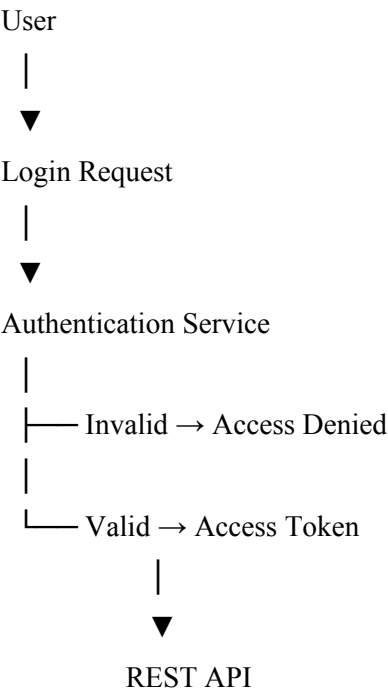
Method	Purpose
GET	Retrieve

	data
POST	Create data
PUT	Update data
DELETE	Delete data

Procedure

- 1. Create a simple cloud application.
- 2. Define REST API endpoints.
- 3. Implement GET and POST operations.
- 4. Add authentication for protected requests.
- 5. Test the API using a browser/API testing tool.
- 6. Deploy the application to a cloud platform.
- 7. Verify the deployed API.

Authentication Flow



Result

A REST API workflow was designed with client communication, authentication, and cloud deployment flow.

EXPERIMENT 3: SERVERLESS EVENT FLOW FOR IMAGE UPLOAD

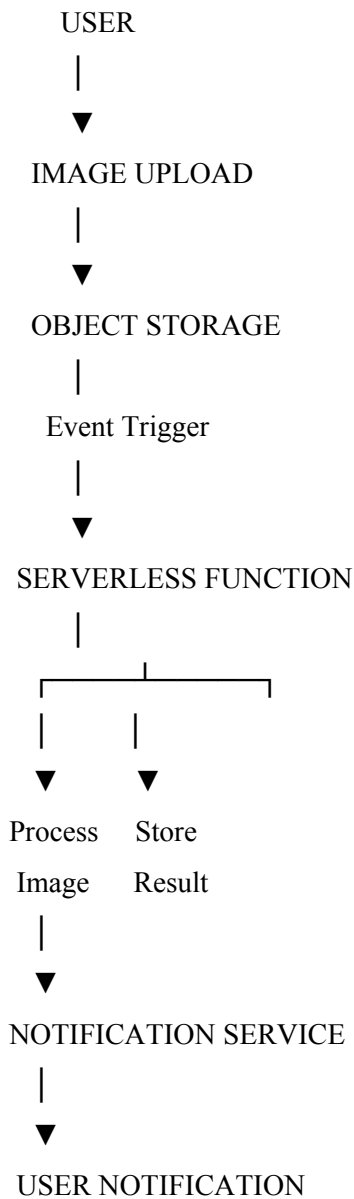
Aim

To design a **serverless event flow** for image upload, processing, storage, and notification.

Objective

- Understand serverless computing.
- Design an event-driven workflow.
- Demonstrate image processing and notification.

Architecture Sketch



Procedure

1. User uploads an image.
2. The image is stored in cloud object storage.
3. The upload generates an event.
4. The event triggers a serverless function.
5. The function processes the image.

6. The processed image/result is stored.
7. A notification is sent to the user.

Advantages

- No direct server management.
- Automatic scaling.
- Event-driven execution.
- Suitable for variable workloads.
- Pay-per-use execution model.

Result

A serverless event-driven architecture was designed for image upload, processing, storage, and notification.

EXPERIMENT 4: EDGE COMPUTING AND FOG COMPUTING

Aim

To compare **edge computing** and **fog computing** based on architecture, latency considerations, and deployment location.

Objective

- Understand edge computing.
- Understand fog computing.
- Compare their deployment locations.
- Analyze latency.

Architecture – Edge Computing

Sensors / Devices

|



EDGE DEVICE

|



CLOUD

Architecture – Fog Computing

Sensors / Devices

|



EDGE DEVICES

|



FOG LAYER



Comparison

Feature	Edge Computing	Fog Computing
Processing location	Near data source	Intermediate layer
Latency	Very low	Low
Architecture	Device/edge → Cloud	Edge → Fog → Cloud
Main purpose	Fast local decisions	Local aggregation and processing
Example	Smart sensor	Regional gateway

Latency Analysis

Traditional Cloud:

Sensor → Internet → Cloud → Response
HIGHER LATENCY

Edge:

Sensor → Edge Device → Response
LOW LATENCY

Fog:

Sensor → Fog Node → Response
LOW LATENCY

Applications:

Edge computing: smart factories, IoT sensors, autonomous systems.

Fog computing: smart cities, distributed IoT networks, regional data processing.

Result

Edge computing provides processing close to the data source, while fog computing provides an intermediate distributed layer between edge and cloud. The choice depends on latency and deployment requirements.

EXPERIMENT 5: CLOUD APPLICATION DEPLOYMENT

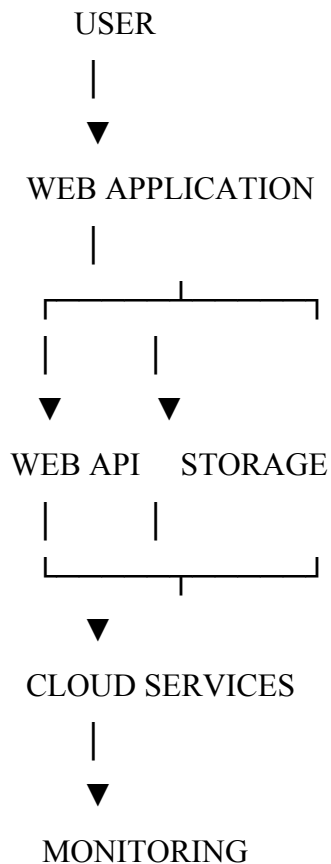
Aim

To demonstrate a cloud application deployment with **user interaction, web API calls, storage access, and monitoring.**

Objective

- Understand cloud application deployment.
- Demonstrate user interaction.
- Use web APIs.
- Access cloud storage.
- Monitor application performance.

Architecture Sketch



Workflow

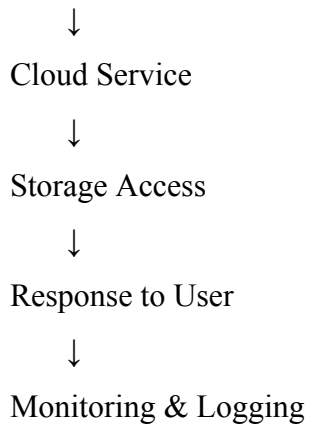
User Interaction



Web Application



Web API Call



Procedure

1. Create a simple cloud application.
2. Provide a user interface.
3. Connect the application to a web API.
4. Configure cloud storage.
5. Test storage access.
6. Deploy the application.
7. Monitor application activity and performance.
8. Check logs for errors or failures.

Components

Component	Role
User Interface	Allows users to interact with application
Web API	Provides application communication
Cloud Storage	Stores application data/files
Cloud Service	Processes application requests
Monitoring	Tracks performance and errors

Result

A cloud application deployment workflow was demonstrated with **user interaction, web API communication, storage access, and monitoring.**